
Two Coding Agents Optimize GraphViz: A Controlled Comparison

Ryan Tran¹, Hailu Xu¹

¹California State University, Long Beach
hailu.xu@csulb.edu

Abstract

Can a coding agent find and fix a real performance bottleneck on its own, with no hint about where the program spends its time? We gave two agents — Claude Code and Codex — the same untouched copy of GraphViz and the same instruction: make the dot layout program faster without changing what it draws. Neither was told which function or phase to look at, and neither could see the benchmark we would judge them on.

Both found the same hot spot: crossing minimization, one of the five phases dot runs. They differ in how far they went. Claude improved three phases and made layout $1.33\times$ to $1.84\times$ faster. Codex improved one phase and ranged from $1.29\times$ faster down to no measurable change at all.

That last result is the interesting one. Codex’s optimization works — it makes that phase $1.31\times$ faster — but on one of our graphs that phase is only 11.5% of the work, so the gain vanishes end to end. How much a phase is worth depends on the shape of the graph, not its size: crossing minimization is 11.5% of layout time on one shape and 64% on another with the same number of nodes. We measured how far apart two runs of the *unmodified* program land, and report only differences larger than that.

1 Introduction

Coding agents are now routinely asked to modify real software. Most evaluations of that ability measure whether an agent can make a failing test pass [6] — a task with a clear oracle, where success is checkable by running the suite. Performance optimization has no such oracle. An agent asked to make a program faster must first decide *what* to make faster, and a plausible-looking change that helps nothing is indistinguishable from a real improvement unless someone measures carefully.

This paper asks a narrow version of that question. Given a mature C codebase, no hint about where time is spent, and an instruction only to make the program faster without changing its output, can a coding agent find a genuine bottleneck and fix it?

We use the dot layout engine from GraphViz [3]. It is real, widely deployed software rather than a benchmark written for the occasion; its layered pipeline [8, 5] has distinct phases whose costs can be measured separately, so a speedup can be attributed rather than merely observed; and its text output is deterministic, which makes “the output did not change” checkable rather than a judgment call.

What we did. Two agents — Claude Code and Codex — each received a pristine checkout of GraphViz pinned to commit 23b06521 and an identical prompt naming no target function, file, or phase. The prompt explicitly forbade the shortcuts that would fake a win: caching layout results, special-casing inputs, and reducing the iteration counts that govern layout quality. Neither agent had access to the evaluation harness or the graphs it uses. We rebuilt each agent’s result ourselves and measured it against an untouched baseline, on three graph shapes with the node count held fixed.

What we found. Both agents independently identified crossing minimization as the target, and both produced byte-identical layout output.

They separate on breadth. Claude improved three phases — ranking, crossing minimization, coordinate assignment — measuring $1.33\times$ to $1.84\times$ end-to-end. Codex improved crossing minimization only, measuring $1.29\times$ down to $0.99\times$: on one shape its optimization is real but the phase is 11.5% of the workload, so nothing survives to the end-to-end result.

Which phase dominates is a property of the graph’s *shape*, not its size. At identical node count, crossing minimization ranges from 11.5% to 64% of layout time, setting a different ceiling on each workload — and on the sparsest one Claude exceeds the ceiling available to any crossing-minimization optimization at all.

Contributions.

- A controlled protocol for evaluating agent-driven optimization: balanced command ordering, which also yields an empirical noise floor from the baseline’s own spread across orderings, and a held-out evaluation harness.
- Phase-resolved measurements of two agents’ work across three graph shapes, with per-phase attribution and the shape-dependent ceiling that bounds it.
- A complete artifact: pinned source, both agents’ diffs and commit histories, raw timing data, and the figure-generation scripts.

Section 2 covers background. Section 3 describes the protocol. Section 4 reports measurements. Sections 5–6 interpret them and state what we did not verify.

2 Background and Related Work

The dot layout pipeline. GraphViz’s dot engine draws directed graphs using the layered approach introduced by Sugiyama et al. [8] and refined by Gansner et al. [5]. It runs five phases in order, each depending on the last: *initialization* parses the file and builds the graph, *ranking* decides which layer each node belongs to, *crossing minimization* orders nodes within a layer to reduce edge crossings, *coordinate assignment* fixes their x positions, and *spline routing* draws the edges.

Ranking and coordinate assignment are both solved with network simplex. That matters later: one agent optimized the shared code and improved both. Crossing minimization is an iterative heuristic.

Agents modifying real code. Benchmarks such as SWE-bench [6] evaluate agents on resolving real repository issues, where a test suite provides the success signal. Performance work differs in that no test can tell you whether an agent found the *right* thing to optimize, and a change that is correct but useless passes every test. Prior work on learned compiler optimization [2] targets transformations chosen in advance; here the choice of target is the object of study.

Prior work on this codebase. An earlier study by some of the present authors applied a multi-model ensemble to the same GraphViz layout code and reported a $3.78\times$ speedup from OpenMP parallelization of crossing minimization [4]. That work has not been peer-reviewed. Our measurements do not reproduce it: Table 6 bounds what any optimization of that phase can yield at 1.11 to $2.58\times$ depending on graph shape, whatever its implementation. We note also that its reported prompt template specifies the target speedup as an input requirement, which makes the reported figure difficult to interpret as an independent measurement. The present paper shares only the codebase and the general question; its protocol, measurements, and conclusions are separate.

3 Method

The design goal was that source code be the *only* thing differing between the binaries we compare. Everything below follows from that.

3.1 Sandboxes and build

Three clones of GraphViz, all at commit 23b06521. One stays untouched as the baseline; the other two go to the agents, each on its own branch so the commit histories survive as artifacts.

All three were built with identical CMake flags (`-DCMAKE_BUILD_TYPE=Release`, Poppler and GVEdit disabled) and installed to separate prefixes. We checked that each dot loads its own build of the layout libraries rather than a shared one.

3.2 The prompt

Both agents received the same text. It stated the goal — make dot faster on large graphs without changing its output — and deliberately named no function, file, or phase. Finding the hot code was part of the task under study. The prompt also:

- described the target workload shape (large layered graphs with heavy edge crossings) without supplying the evaluation graphs, so agents generated their own test inputs;
- required byte-identical `-Tdot` output and pointed at the 260 graphs in the repository’s own `tests/` directory;
- enumerated disallowed shortcuts: detecting or special-casing specific inputs, caching layout results across runs, reducing iteration counts or loosening convergence thresholds, hardcoding computed values, and modifying the test suite;
- stated that measurement would be independent, that identical builds vary by several percent on the target machine, and that a modest honest result or no result at all was an acceptable outcome.

The iteration-count prohibition matters most. Crossing minimization is iteration-bounded, so cutting its limit yields a large, real, reproducible speedup that is not an optimization. The prompt asked for that to be reported as a finding rather than shipped as a change.

Agents worked autonomously with no turn or time limit, committing incrementally and writing a report of what they profiled, changed, abandoned and verified. Neither had access to the evaluation harness, the benchmark graphs, or the other agent’s sandbox.

3.3 Evaluation graphs

Graph generation is deterministic. Nodes are split into equal layers and each points at `fanout` random nodes in the next layer, which is what creates the crossings layout has to resolve.

Node count is fixed at 5,000 and only the shape varies (Table 1):

Table 1: Evaluation graphs. All have 5,000 nodes. **Ranks:** $5,000 \div \text{nodes-per-rank}$. **Edges:** $(\text{ranks} - 1) \times \text{nodes-per-rank} \times \text{fanout}$.

Shape	Nodes/rank	Fanout	Ranks	Edges
sparse-deep	10	2	500	9,980
default	100	3	50	14,700
dense-shallow	200	4	25	19,200

These parameters keep every phase measurable. Wider ranks make crossing minimization so expensive that nothing else is visible next to it.

The generator does not pin ranks. Wrapping each layer in `rank=same` is legal DOT, but it supplies the answer the ranking phase exists to compute: on otherwise identical graphs that reduces ranking from 267 ms to 1 ms and removes the phase from measurement.

3.4 Measurement

All timings use `hyperfine` 1.20.0 [7] with 10 warmup and 10 timed runs. All binaries for a graph are measured in one invocation, seconds apart under the same thermal conditions; separate invocations produce order-dependent bias large enough to swamp the effects we are looking for.

`hyperfine` runs all repetitions of one command before starting the next, so a fixed order always hands the same binary the tail of any drift. We rotate the order so each binary occupies each position once, then pool: three orderings, 30 timed runs per binary per graph.

The rotation also supplies the noise floor. The baseline is measured three times, once in each position, so the spread among its own means is what the protocol reports for a program compared against itself.

Two conventions hold throughout this paper. A *mean* is always the arithmetic mean of individual timed runs, with warmup runs excluded. A *speedup* is always the ratio of two such means — never the mean of per-run or per-ordering ratios. The two are not equal, and every ratio we report is the former.

We prefer this to benchmarking separately-built identical binaries, which we also tried. Those would be measured in their own invocation, with a different command set and a different thermal profile from the runs they are meant to bound. The baseline’s spread is measured in the same invocations, on the same graphs, against the same competitors as the results it guards.

3.5 Phase instrumentation

Phase costs are measured inside the process. An identical patch — applied by cherry-pick, so all three diffs match byte for byte — reads a monotonic clock around each phase and writes the elapsed time to standard error. It is off unless an environment variable is set, so layout output is unchanged and the correctness check below is unaffected.

Parsing and output writing happen outside the layout routine, so the phases do not sum to wall clock. We report the difference rather than assigning it to a phase.

3.6 Correctness checking

Correctness is checked on `dot -Tdot` output, not a rendered image. The text format is deterministic, so byte-identity means something; image comparison would fold in rendering and floating-point behaviour we are not studying. We also read both diffs against the prompt’s list of disallowed shortcuts.

3.7 Platform

Experiments ran on an Apple M4 Pro (12 cores: 8 performance, 4 efficiency), 24 GB unified memory, macOS 26.5.2, with Apple Clang 17.

4 Results

4.1 Noise floor

Because command order rotates, the baseline is measured once in each position. Table 2 reports how far apart its three means land — the apparent speedup this protocol produces when nothing has changed.

Worked example, crossing minimization on sparse-deep: the three per-ordering means are 61.47, 62.07 and 61.60 ms, giving pairwise ratios 1.010, 1.002 and 1.008. The largest is the cell.

The floor is not uniform, which is why we report it per graph and per phase. Initialization on default is the noisiest cell at $1.119\times$ — six times the crossing-minimization floor on the same graph — because it is parsing and setup, sensitive to page cache. Pooling would let that one cell set a threshold everywhere.

Three ratios per cell is a small sample, so each is a rough bound. That is the cost of not conflating graphs.

Table 2: Apparent speedup measured between the baseline and itself, per graph and per phase. **Each cell:** average the baseline’s 10 timed runs within each of the three command orderings, giving three means; form all three pairwise ratios, larger divided by smaller; the cell is the largest of the three.

Measurement	sparse-deep	default	dense-shallow
End-to-end	1.006×	1.019×	1.006×
Initialization	1.018×	1.119×	1.060×
Ranking	1.020×	1.015×	1.003×
Crossing minimization	1.010×	1.015×	1.006×
Coordinate assignment	1.019×	1.012×	1.012×
Spline routing	1.010×	1.007×	1.005×

Throughout the rest of this section, we treat a measured ratio as a change only if it exceeds the floor for that same phase and that same graph. Ratios that do not are marked in the tables that follow.

This floor is measured inside the same hyperfine invocations as the results it bounds, on the same graphs, against the same competing commands. The more conventional construction — benchmarking separately-built identical binaries — places them in an invocation of their own, with a different command set and thermal profile, and so describes a different experiment from the one being guarded.

4.2 Phase weights depend on shape, not size

Which phase dominates is not fixed. At identical node count, crossing minimization takes 11.5% of layout time on one shape and 64.0% on another, while coordinate assignment moves the other way, 42.1% down to 9.4%. This sets a different ceiling on each workload, and it is why one optimization gives different end-to-end results on different graphs.

Table 3 and Figure 1 give the baseline cost of each phase.

Table 3: Baseline phase cost. Phases are timed inside the process, so these are direct measurements rather than differences. **ms:** mean of that phase over all 30 timed runs (10 runs × 3 command orderings). **share:** that phase’s ms divided by the sum of the five phase ms. **Unaccounted:** wall clock minus that sum — parsing and output writing, which happen outside the layout routine.

Phase	sparse-deep		default		dense-shallow	
	ms	share	ms	share	ms	share
Initialization	148.0	27.7%	147.0	11.2%	155.4	5.2%
Ranking	76.4	14.3%	266.9	20.4%	537.0	18.0%
Crossing minimization	61.7	11.5%	575.0	43.9%	1907.8	64.0%
Coordinate assignment	225.4	42.1%	251.5	19.2%	280.8	9.4%
Spline routing	23.2	4.3%	68.3	5.2%	101.6	3.4%
Measured total	534.9	100%	1308.7	100%	2982.6	100%
Wall clock	600.2		1412.9		3118.3	
Unaccounted	65.3	10.9%	104.2	7.4%	135.7	4.4%

We report the unaccounted row rather than folding it into a phase. It is largest on sparse-deep, where the layout work is smallest and the fixed cost of parsing shows through.

4.3 End-to-end results

Table 4 and Figure 2 give layout time for all three binaries on each shape.

Claude improves every shape, 1.33× to 1.84×. Codex ranges from 1.29× down to 0.99× — on sparse-deep, indistinguishable from the unmodified baseline.

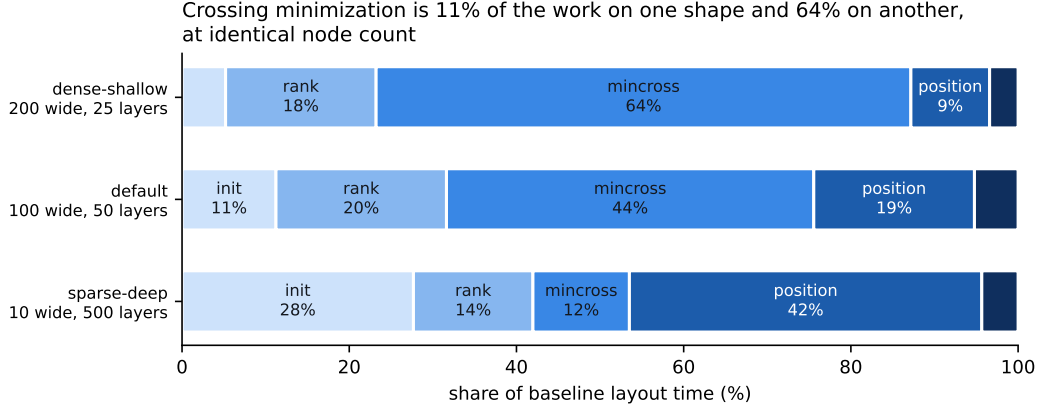


Figure 1: Baseline runtime composition by shape, at identical node count.

Table 4: End-to-end layout time. **Mean and sd:** over all 30 timed runs (10 runs $\times$ 3 command orderings). **Speedup:** baseline mean divided by that binary’s mean. A dagger marks a ratio that does not exceed the end-to-end floor for that graph in Table 2; we read those as no change.

Graph	Baseline	Claude	Codex
sparse-deep	600.2 $\pm$ 15.7	450.4 $\pm$ 30.2 (1.33$\times$)	605.6 $\pm$ 40.2 (0.99 $\times$) [†]
default	1412.9 $\pm$ 26.2	943.1 $\pm$ 33.4 (1.50$\times$)	1215.3 $\pm$ 20.6 (1.16$\times$)
dense-shallow	3118.3 $\pm$ 38.8	1693.8 $\pm$ 26.1 (1.84$\times$)	2417.5 $\pm$ 89.7 (1.29$\times$)

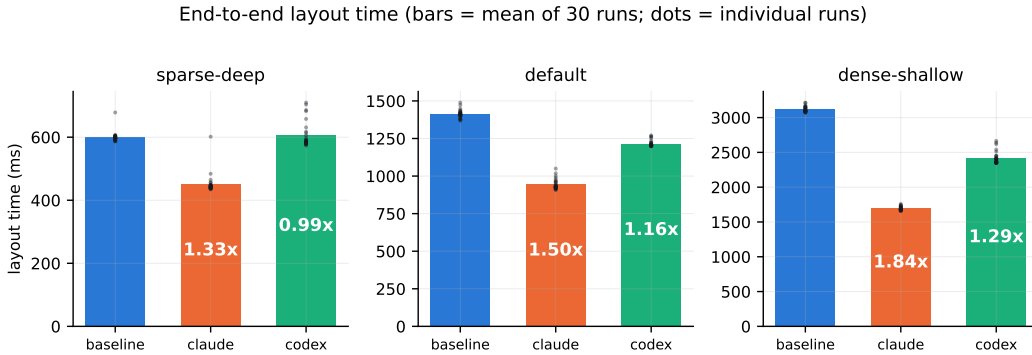


Figure 2: End-to-end layout time on all three shapes.

Five of the six ratios clear their graph’s floor by a wide margin; the smallest surviving is Codex’s $1.16\times$ on default against a floor of $1.019\times$. Codex on sparse-deep clears nothing, and we report it as no improvement rather than a small one.

Per-ordering figures rule out an artifact: Claude measures 1.31–1.35, 1.46–1.53 and 1.82–1.86 across the three orderings, Codex 0.93–1.03, 1.15–1.17 and 1.24–1.32.

4.4 Per-phase attribution

Table 5 and Figure 3 break the end-to-end result down by phase.

Table 5: Per-phase speedup over baseline. **Each entry:** the baseline’s mean for that phase divided by that binary’s mean for the same phase, each mean taken over all 30 timed runs. A dagger marks an entry that does not exceed the floor for that same phase and graph in Table 2; those are no change, not small effects.

Phase	sparse-deep		default		dense-shallow	
	Claude	Codex	Claude	Codex	Claude	Codex
Initialization	0.99 [†]	0.94 [†]	1.00 [†]	1.01 [†]	1.02 [†]	0.99 [†]
Ranking	2.46	0.98 [†]	2.04	1.00 [†]	2.05	0.98 [†]
Crossing minimization	1.47	1.31	1.87	1.52	2.34	1.61
Coordinate assignment	1.62	0.97 [†]	1.37	1.00 [†]	1.25	0.98 [†]
Spline routing	1.00 [†]	1.00 [†]	0.99 [†]	0.99 [†]	1.00 [†]	0.99 [†]

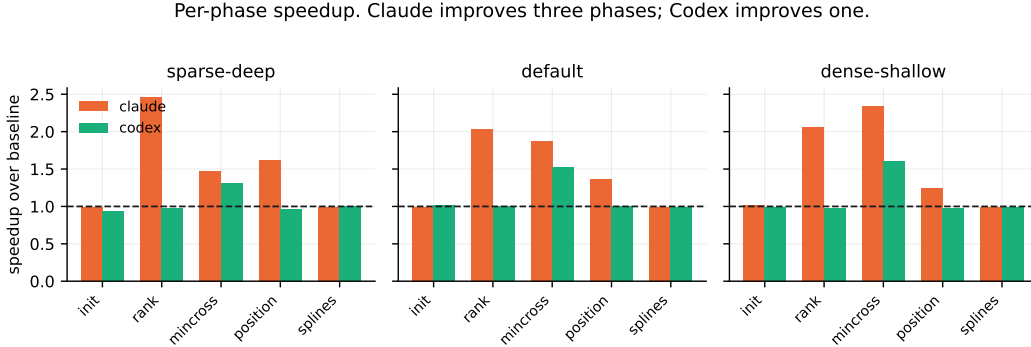


Figure 3: Per-phase speedup. Claude improves ranking, crossing minimization and coordinate assignment on every shape; Codex improves crossing minimization only.

Twelve of the 30 entries exceed their floor: Claude on ranking, crossing minimization and coordinate assignment across all three graphs, and Codex on crossing minimization across all three. Nothing sits near a boundary — the smallest survivor is $1.25\times$ against a $1.012\times$ floor, the largest marked entry $1.02\times$ against $1.060\times$.

So Claude improves three phases: ranking by roughly $2\times$ everywhere, crossing minimization by 1.47 to $2.34\times$, coordinate assignment by 1.25 to $1.62\times$. Codex improves crossing minimization by 1.31 to $1.61\times$ and nothing else.

That is what produces its $0.99\times$ on sparse-deep. The optimization is real at $1.31\times$ on the phase, but the phase is 11.5% of that workload.

Neither agent touches initialization or spline routing, as expected: both worked in `mincross.c`, and Claude also in `ns.c`.

4.5 Correctness

Both binaries produce `dot -Tdot` output byte-identical to the baseline on every evaluation graph, with instrumentation both enabled and disabled.

We also ran the 260 graphs in GraphViz’s own test suite through both binaries and compared against the baseline byte for byte. Both agents changed `mincross.c`, which only `dot` uses; Claude also changed `ns.c`, which `neato` reaches as well, so we checked both engines. All 1,040 comparisons are identical, with matching exit codes.

Reading the diffs against the prompt’s disallowed shortcuts found no reduced iteration counts, early exits, input special-casing or modified tests, and no concurrency of any kind.

5 Discussion

5.1 Both agents found the bottleneck; only one kept going

Neither agent was told where to look, and both went to crossing minimization. Working in isolation, both arrived at the same transformation: fusing the two symmetric crossing-count passes. Locating the expensive phase is evidently not the hard part.

What separates them is how far they went next. Codex stopped at the phase it identified. Claude continued into network simplex, which serves both ranking and coordinate assignment, and that accounts for its entire margin — the same pattern on three workloads with very different composition, not an inference from one measurement.

5.2 Why breadth mattered: the ceiling moves with shape

Amdahl’s law [1] bounds what optimizing one phase can yield, and that bound depends on the phase’s share of runtime — which Table 3 shows is a property of the graph’s shape.

Table 6: What optimizing crossing minimization alone can achieve, against what each agent measured. **Ceiling:** baseline wall clock divided by (wall clock minus that phase’s measured cost), i.e. the phase driven to zero. **Claude and Codex:** the end-to-end speedups from Table 4, carrying the same dagger convention.

Graph	Ceiling, crossing min. only	Claude	Codex
sparse-deep	$1.11\times$	$1.33\times$	$0.99\times^\dagger$
default	$1.69\times$	$1.50\times$	$1.16\times$
dense-shallow	$2.58\times$	$1.84\times$	$1.29\times$

Codex sits below the ceiling on all three shapes, as an agent that optimized only that phase must. On sparse-deep the ceiling is $1.11\times$, so even a perfect crossing-minimization optimization would be nearly invisible end-to-end. Codex’s $0.99\times$ is not a failed optimization — it works, at $1.31\times$ on the phase — but a phase worth 11.5% of the workload.

Claude *exceeds* that ceiling, $1.33\times$ against $1.11\times$, which is only possible by improving other phases. That is the breadth argument at its sharpest: on that workload no amount of further work on the phase both agents found could have matched it.

The ceiling an optimization can reach is a property of the input, not the program, and a single benchmark graph cannot show that.

5.3 Cost

Table 7: Claude run cost, as reported by its session log.

Metric	Value
Model	claude-opus-5
Working time	4 h 27 m 3 s
Input tokens	730
Output tokens	620,448
Cache creation tokens	1,270,669
Cache read tokens	89,851,473
Total tokens	91,743,320

Table 8: Codex run cost, as reported by its session log. Cached input is a subset of input; reasoning output is a subset of output.

Metric	Value
Model	gpt-5.6-terra
Working time	23 m 42 s
Input tokens	5,114,035
Cached input tokens	4,985,600
Output tokens	20,826
Reasoning tokens	4,555
Cache-write tokens	0
Total tokens	5,134,861

6 Limitations

We state these plainly because several of them bound the conclusions above.

One run per agent. Each agent was invoked once. Agent behaviour is stochastic, so we cannot say whether the ordering we observed is stable or whether a second invocation of either would find a different target. This is the single weakest point in the study, and it means our results describe two particular runs rather than two systems.

Synthetic graphs only. All three shapes come from the same generator, which produces uniform rank widths and constant fanout. Real inputs to dot are irregular in both. A generator also encodes assumptions about the workload that do not show up in aggregate timings, and we cannot rule out ones we have not noticed.

Reproducibility Statement

Every number in this paper comes from data in the artifact below. No result is estimated, extrapolated, or reported from a source other than the raw measurement files.

Artifacts.

- Claude’s diff and commit history:
<https://github.com/harvest7777/claude-graphviz-run/compare/main...agent-work>
- Codex’s diff and commit history:
<https://github.com/harvest7777/codex-graphviz-run/compare/main...agent-work>

- Raw timing data and session dumps:
<https://github.com/harvest7777/graphviz-results-dump>

Source. GraphViz at commit 23b06521ab916e7f4f2ae2f9ef8413a8ccc66601, from <https://gitlab.com/graphviz/graphviz.git>. Both agent branches are named `agent-work` and `branch` from that commit.

Build.

```
cmake -S <sandbox> -B <build> \
  -DCMAKE_INSTALL_PREFIX=<prefix> \
  -DWITH_POPPLER=OFF -DWITH_GVEDIT=OFF \
  -DCMAKE_BUILD_TYPE=Release
cmake --build <build> -j12 && cmake --install <build>
```

Identical for all three sandboxes. On macOS a Bison ≥ 3.0 path must be passed explicitly, as the system Bison is too old for the GraphViz build.

Measurement. `hyperfine` 1.20.0, 10 warmup and 10 timed runs, all binaries for a given graph in a single invocation, repeated over three command orderings so each binary occupies each position once. Graphs are produced by the fixed-seed generator in the artifact repository. Raw per-run timings for every configuration are in the results repository as `hyperfine` JSON exports, alongside the per-run phase logs.

Figures. Every figure is regenerated from the raw JSON by a single script in the artifact (`figures/make_figures.py`).

Environment. Apple M4 Pro, 12 cores (8 performance, 4 efficiency), 24 GB unified memory, macOS 26.5.2, Apple Clang 17.0.0.

Phase instrumentation. The patch that reports per-phase timings is a single commit on `lib/dotgen/dotinit.c`, applied by cherry-pick to all three sandboxes so that the three diffs are byte-identical. It is inactive unless `GV_PHASE_TIMING` is set in the environment.

Agents. `claude-opus-5` (Claude Code) and `gpt-5.6-terra` (Codex), each invoked once, autonomously, with the prompt reproduced in the artifact repository. Neither had network access to the evaluation harness or benchmark graphs.

7 Conclusion

Two coding agents, given an identical checkout of GraphViz and a prompt naming no target, both located crossing minimization as the expensive phase and both produced output-preserving improvements to it. Finding the bottleneck was evidently not the difficult part of the task.

What separated them was breadth. Claude carried on into network simplex, which serves two further phases, and improves three of five on every shape we tested — $1.33\times$ to $1.84\times$ end-to-end. Codex optimized the one phase it identified and measures $1.29\times$ down to $0.99\times$. On the sparsest shape, Codex’s optimization works, at $1.31\times$ on its phase, and still yields nothing end-to-end because that phase is 11.5% of the workload.

The ceiling an optimization can reach is a property of the input, not of the program. At fixed node count, crossing minimization ranges from 11.5% to 64% of layout time across our three shapes, and on the sparsest one Claude exceeds what any crossing-minimization optimization could achieve. A single benchmark graph cannot show this.

The results are modest and we have tried to report them at the size they are. The protocol may be the more durable contribution: balance command ordering and take the floor from the baseline’s own spread across those orderings; keep the evaluation harness out of the agents’ reach; forbid the shortcuts that manufacture speedups; measure phases directly rather than by subtraction. Each caught something a simpler design would have reported as a finding.

Immediate future work is to repeat both agent runs to establish whether the ordering is stable, extend to a second graph size and to non-synthetic inputs, and complete the correctness checks named in Section 6.

References

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.
- [2] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. Large language models for compiler optimization, 2023. arXiv preprint.
- [3] John Ellson, Emden Gansner, Lefteris Koutsofios, Stephen C. North, and Gordon Woodhull. Graphviz — open source graph drawing tools. In *Graph Drawing (GD 2001)*, LNCS 2265, pages 483–484, 2001.
- [4] Edwyn Estrada, Hailu Xu, and Chunhua Liao. Learning to optimize: Automated parallel code generation for Graphviz dot layout algorithms, 2025. Unpublished manuscript. OpenReview submission <https://openreview.net/forum?id=88zyE3fJzQ>.
- [5] Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Kiem-Phong Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230, 1993.
- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [7] David Peter. hyperfine: A command-line benchmarking tool, 2023. Version 1.20.0. <https://github.com/sharkdp/hyperfine>.
- [8] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981.